Performance Engineering of a PySpark Pipeline on NYC Taxi Trips

Gabriel Matias Falcão (77390)
NOVA School of Science and Technology, Universidade NOVA de Lisboa
Lisbon, Portugal

ABSTRACT

This report presents the profiling and optimisation of a PySpark pipeline that processes 29.7 M NYC yellow-taxi trips (three mandatory operations plus origin-destination flows and trip-type classification). Each report's structure follows a profile, hypothesise, optimise, measure loop, sort-merge joins against tiny tables, spills in under-partitioned reduce stages and repeated scans are identified as the main weights of the baseline. Optimisations include broadcast joins, writing results once, materialising the cleansed data and explicitly partitioning a cached result, which cut individual operations by up to 5×; caching the wrong DataFrames and smaller scan splits are reported as measured failures. Execution time scales sub-linearly with input size because of a fixed cost (~39 s on the Docker cluster, ~25 s locally), and a single-machine Docker cluster shows why adding executors on shared hardware does not help.

KEYWORDS

Apache Spark, performance engineering, query plans, broadcast join, caching

1 INTRODUCTION

The pipeline processes the NYC Taxi and Limousine Commission (TLC) yellow taxi trip records [3]: one parquet file per month plus a 265-row zone lookup table. Development and all measurements use 2026-01..08 (29.7 M records, 488 MB of parquet). Besides the mandatory operations (M1 cleaning statistics, M2 temporal aggregation per borough, day and hour, M3 monthly zone statistics with rolling windows), two optional ones were chosen for the bottlenecks they expose. **O2** (origin-destination flows) joins the zone table twice and derives three outputs (route shares, top-20 per borough, Q1/Q3 reversals) from one monthly aggregate, so it stresses multi-way joins and recomputation. **O4** (trip-type classification) aggregates trips into at most 263×24 buckets and joins that small result back to every trip, the shape where join strategy and hot-zone skew decide the cost.

Every operation joins ~21 M trips with a tiny table and aggregates, so the cost is how Spark moves and re-reads data, not arithmetic. Code, logs and per-optimisation reports are in the repository [2].

2 PIPELINE

All operations share two steps. *Load* groups the files by physical parquet schema (2026 has two variants), casts each group to the documented schema with `try_cast` and combines them with `unionByName`. *Clean* keeps a record only if it passes the M1 rules: no null in a used column, fare and distance > 0, 1–6 passengers, dropoff after pickup, location ids in 1–263 (20.95 M records remain). Each operation then joins the zone table and aggregates; M3 and O2 add windows, O2 a pivot, O4 a second aggregation after the join back.

Op	Wall (s)	Stages	Tasks	Shuffle read	Spill mem / disk
M1	14.5	8	76	5.4 KB	0
M2	45.7	18	152	953.8 MB	1.9 GB / 832.6 MB
M3	21.1	9	76	318.5 MB	768 MB / 261 MB
O2	20.3	81	515	53.9 MB	0
O4	22.1	18	220	222.4 MB	704 MB / 107 MB

Table 1: Baseline, median of 3 runs (stage, task and shuffle counts identical in all runs; `reports/baseline` [2]).

Design decisions: the DataFrame API is used throughout, because operations are composed in Python from shared pieces (cleaning rules, zone joins, feature flags), and it compiles to the same Catalyst plans as SQL; no UDFs are used, so no work goes through Python workers; timestamps are read as `timestamp_ntz` with the session zone fixed to UTC, so hours and weekdays do not depend on the host; routes and zones are keyed by id because zone names are not unique. In the baseline `spark.sql.autoBroadcastJoinThreshold=-1`, so every join starts as a Sort-Merge Join (SMJ) and each optimisation is an explicit, measured feature flag. Figure 1 shows the physical plan of M2.

3 PROFILING AND BASELINE

Setup. Laptop with an Intel i7-1195G7 (4 cores / 8 threads), 15 GiB RAM; Spark 4.2.0 (PySpark), OpenJDK 25, Python 3.14, `local[*]` (8 cores), `spark.driver.memory=2g`; non-default configuration: no automatic broadcast, session time zone UTC. Adaptive Query Execution (AQE) [7] is on with the default 200 shuffle partitions.

Method. Each configuration runs 3 times; reported times are medians. Outputs go to Spark's `noop` sink, because `show()` computes only the rows it prints. Every run logs `explain("formatted")` and `explain(True)`, and the Lab 3 profiler records jobs, stages, tasks, shuffle and spill per operation. Per-stage task-time quantiles come from the Spark UI REST API[5]. Spark's input metric reports 6 MiB for scans that read 383 MiB of parquet column chunks, so input volume is taken from the parquet footers instead.

Bottlenecks.[1] (1) *Trip-side shuffle of the SMJ*: every operation hashes and sorts all ~21 M cleaned trips to join a table of $\leq 6\,312$ rows (Figure 1, left). The stage after that shuffle is the most expensive of each operation and holds all of the spill (M2 stage 14: 476 MB read, 896 MB / 392 MB spilled, 99 s of task time). (2) *Too few reduce partitions*: AQE coalesces the 200 shuffle partitions into 9, so each task sorts ~53 MB and spills, and 9 tasks run on 8 cores in two waves, the 9th alone for ~12 s. (3) *Recomputation*: M2's display and parquet write are two actions, so its scan and join run twice

[1] Logs: `reports/baseline` [2].

```
HashAggregate (final)
+- Exchange (Borough, day, hour)
   +- HashAggregate (partial)
      +- SortMergeJoin PULocationID=LocationID
         :- Sort
         :  +- Exchange (PULocationID) <- all trips
         :     +- Union (2 schema groups)
         :        :- Project(try_cast) / Filter
         :        :  +- Scan parquet [7 cols]
         :        +- Project(try_cast) / Filter
         :           +- Scan parquet [7 cols]
         +- Sort
            +- Exchange (LocationID)
               +- Scan csv (zones)
```

```
HashAggregate (final)
+- Exchange (Borough, day, hour)
   +- HashAggregate (partial)
      +- BroadcastHashJoin BuildRight
         :- Union (2 schema groups)
         :  :- Project(try_cast) / Filter
         :  :  +- Scan parquet [7 cols]
         :  +- Project(try_cast) / Filter
         :     +- Scan parquet [7 cols]
         +- BroadcastExchange
            +- Scan csv (zones)
```

Figure 1: M2 physical plan (`explain("formatted")`, simplified). Left: baseline, every cleaned trip is shuffled and sorted to join 265 rows. Right: after broadcasting the zone table (Section 4.1).

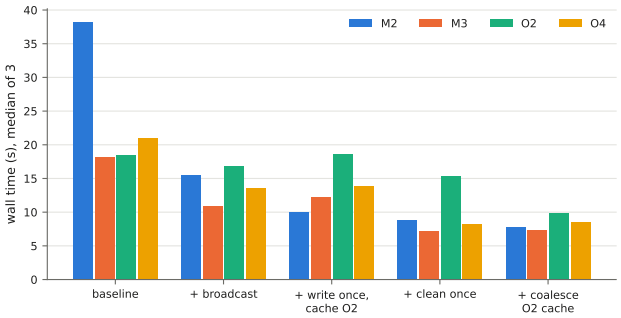


Figure 2: Wall time per operation across the cumulative optimisations, one session, median of 3 runs of all operations (`reports/cumulative` [2]).

(stages 11/14 rerun as 20/23, half of its shuffle); O2 rescans the trips for each of its 3 outputs and O4 scans them twice. (4) *Skew*: not found, the slowest reduce task is at most 1.1× the median, because AQE packs many zones into each partition. (5) *Column pruning*: already done by Catalyst, which pushes the projection into the scan (7 of 20 columns read); selecting the used columns explicitly (`--prune-column`) leaves the `ReadSchema` unchanged. Three of the seven columns are read only for the cleaning rules, which Section 4.3 removes.

4 OPTIMISATION STUDY

Optimisations are cumulative: each one is measured against the configuration that includes the previous ones, with all other settings unchanged. Table 2 summarises them, and Figure 2 re-measures the whole path in a single session (3 runs per configuration, each started on an idle machine), so the steps are directly comparable. It also shows that caching O2 alone did not pay off inside a full run (16.9 s → 18.6 s); only the coalesced cache did (9.8 s).

4.1 Broadcast the small side of every join

Bottleneck: the trip-side `Exchange` and sort of the SMJ and the spill after it (Section 3). *Hypothesis:* the zone table (265 rows) and O4's bucket table (≤ 6 312 rows) fit in every executor, so a Broadcast Hash Join (BHJ) can probe them inside the scan tasks and the trips never need to be shuffled. *Change:* `--broadcast-join` wraps the small side in a `broadcast()` hint [7]. *Result:*[2] every `SortMergeJoin` becomes a `BroadcastHashJoin` (Figure 1, right), the 9-task reduce stage disappears, and only partial aggregates are shuffled. Shuffle read falls from 953.8 MB to 1.5 MB for M2 (318.5 MB to 0.7 MB for M3, 222.4 MB to 1.8 MB for O4), all spill goes to zero, and M2, M3 and O4 run 2.2×, 1.8× and 1.4× faster. O2 does not change, because its joins run on aggregated routes rather than trips.

4.2 Reusing results: cache versus write once

Bottleneck: with the shuffles gone, the scans dominate, and multi-action operations scan the trips once per action (M2 twice, O2 thrice, O4 twice). *Hypothesis:* persisting the shared DataFrame leaves one scan per operation. *Change:* `--cache-dataframe` caches M2's result, O2's monthly routes and O4's cleaned trips. *Result:*[3] only O2 improves (21.7 s → 17.0 s, 1.28×): its cache is a 4.8 MiB aggregate, so two of three scans disappear cheaply. M2 gets slower (20.9 s → 23.2 s): the cached result keeps all 200 shuffle partitions, which AQE would otherwise coalesce into one, so the write runs 200 tasks and produces 663 files instead of 6. O4 also gets slower (15.6 s → 19.9 s): building a 390 MiB in-memory cache of 21 M trips adds 41 s of task time to the scan, more than the second parquet scan it replaces. The cache was therefore kept for O2 only. For M2 the fix is to write once (`--write-once`)[4]: write the parquet output first and display the 967-row result read back from it. One of the two 18-task scan stages disappears, task time drops by 37 % and M2 goes from 23.0 s to 15.3 s (1.5×), with identical output.

4.3 Clean once, read many

Bottleneck: every operation still starts with a `Union` of two scans, each with `try_cast` projections and a 13-predicate cleaning filter; a run of all operations does this 9 times on the same rows and also reads columns used only by the cleaning rules. *Hypothesis:* materialising the cleaned, typed trips once as parquet removes that work from every operation, at the cost of one write; in-memory caching was ruled out by the O4 result and by the 2016–2020 data size. *Change:* `--materialise-clean` writes the cleaned trips (9 columns, 8 MiB row groups, 324 MiB) and runs M2–O4 on that copy; M1 still reads the raw files. *Result:*[5] `Union` nodes drop from 9 to 1 and parquet scans from 18 to 10; M2's scan reads 5 columns

[2] Logs: `reports/all/broadcast-join` [2].
[3] Logs: `reports/all/reuse-results` [2].
[4] Logs: `reports/M2/write-once` [2].
[5] Logs: `reports/all/materialise-clean` [2].

Optimisation	Bottleneck addressed	Measured impact (median wall time)
Broadcast joins	Trip shuffle + sort of the SMJ, spill	M2 45.7⟶20.9 s, M3 21.1⟶11.8 s, O4 22.1⟶15.6 s; spill 1.9 GB⟶0
Cache shared results	Trips scanned once per action	O2 21.7⟶17.0 s; M2 and O4 slower, kept for O2 only
Smaller scan splits	Uneven scan tasks	No gain, parallelism capped by 32 row groups
Write once (M2)	Result computed twice	M2 23.0⟶15.3 s, one scan stage less
Clean once, read many	Re-scan, cast and clean per operation	M2 13.8⟶9.0 s, M3 13.4⟶6.8 s, O2 19.0⟶16.3 s, O4 13.9⟶7.9 s; all ops 96.2⟶92.3 s incl. write
Coalesce O2 cache	200 tiny tasks per cache read	O2 24.2⟶14.8 s (run alone), tasks 2 443⟶370
All applied (one session)		M2 38.3⟶7.7 s (5.0×), M3 18.2⟶7.4 s (2.5×), O2 18.4⟶9.8 s (1.9×), O4 21.0⟶8.6 s (2.4×); all ops 136.8⟶76.9 s incl. clean write

Table 2: Summary of the optimisations; each row is measured against the configuration with the rows above it applied, in its own report. The last row comes from one session that ran every cumulative configuration 3 times (Figure 2).

with one pushed filter instead of 7 with 13. Task time falls by 41 % (M2), 60 % (M3), 19 % (O2) and 48 % (O4). The one-time write costs 19.4 s, yet a run of all operations already drops from 96.2 s to 92.3 s including it, and to 72.5 s once the copy exists. A `DISK_ONLY` persist of the same data was also measured: it builds 4.4 s slower (21.4 s versus 17.0 s), reads are within noise, and unlike the parquet copy it is lost when the application ends.

4.4 Partition the O2 cache explicitly

Bottleneck: O2 is now the slowest operation, and its time is mostly fixed: 10.6 s on one month of data, 15.8 s on eight (Section 5). Its three outputs read the 4.8 MiB route cache in five 200-task stages whose tasks take 12–132 ms (median); these stages hold 74 s of O2's 128 s of task time. *Hypothesis:* the cache freezes the 200 shuffle partitions of the window that produces the routes, and AQE coalesces partitions when reading a shuffle but not when reading a cache, so each output pays task overhead for ~25 KB partitions (the same mechanism that slowed M2's cache in Section 4.2). *Change:* `--coalesce-cache` coalesces the routes to the default parallelism (8) before caching, without a shuffle. *Result:*[6] tasks drop from 2 443 to 370, the five stages run 8 tasks of 0.3–1.7 s with 39 s instead of 74 s of task time, and O2 goes from 24.2 s to 14.8 s (1.64×, O2 run alone) with identical output. Smaller scan splits (`maxPartitionBytes`) gave no gain[7]: the raw files hold only 32 row groups, which splits cannot divide [4, 6]. Lowering `spark.sql.shuffle.partitions` from 200 to 8 (`--shuffle-partitions`)[8] also gave no gain beyond noise: AQE already coalesces the shuffles, so tasks change only in O2 (370⟶341, 10.5⟶9.2 s).

5 ANALYSIS

No multi-node cluster was available, so all cluster results come from a Docker standalone cluster [8] on the same laptop (one master, workers of 2 cores / 2 GiB, driver in its own container), measured in a single session with the final configuration[9]; `local[*]` runs supplement them.

Dataset scale. On 3 workers, the pipeline was run on 10, 20 and 50 % samples (the method suggested by the assignment) and on 1, 2, 4 and 8 month files (Figure 3). Time grows sub-linearly: 8× more input costs only 2.3× more time (47.2 s ⟶ 110.7 s). A straight line through the month series, $T \approx 39\,\text{s} + 73\,\text{s} \cdot f$, shows that about 39 s of every run is fixed: scheduling the ~10 actions per run on the executors, their warm-up, and O2's small cached stages. Locally the same fit gives $25\,\text{s} + 66\,\text{s} \cdot f$[10], so the cluster adds mostly fixed cost. The data-dependent part is linear, because after the optimisations no trip rows are shuffled. Sampling understates small inputs: `sample()` drops rows after the scan, so a 10 % sample (55.9 s) is slower than a real 12.5 % input (47.2 s).

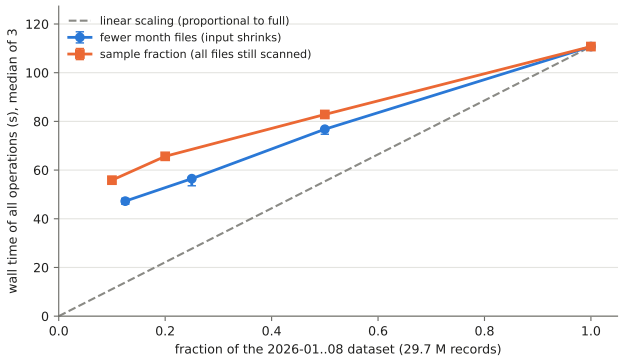


Figure 3: Wall time of all operations versus dataset fraction (median of 3, bars: min–max), final configuration, 3-worker Docker cluster (`reports/docker-session` [2]).

Workers	Cores	Wall (s), median [min–max]	Speedup	Efficiency
1	2	85.7 [84.2–86.7]	1.00	1.00
2	4	92.5 [91.0–93.7]	0.93	0.46
3	6	110.2 [108.8–110.8]	0.78	0.26

Table 3: Docker standalone cluster on one laptop (4 cores / 8 threads), all operations, 3 runs per size. Speedup and efficiency relative to 1 worker. For reference, `local[*]` takes 76.9 s (`reports/docker-session` [2]).

Cluster size. With the full data (Table 3), every added worker makes the run slower. The extra cores are not extra hardware: the same work costs more task time as executors are added (M3 13 s with one worker, 39 s with three), because

6 executor cores, driver and master compete for 4 physical cores, the clock drops with more active cores, and every executor Java Virtual Machine (JVM) repeats class loading and JIT compilation

[6] Logs: `reports/02/coalesce-cache` [2].
[7] Logs: `reports/all/split-size` [2].
[8] Logs: `reports/all/shuffle-partitions` [2].
[9] Logs: `reports/docker-session` [2].
[10] Logs: `reports/scalability` [2].

Op	Baseline (s)	Final (s)	Speedup
M2	44.0	13.8	3.2×
M3	19.1	10.4	1.8×
O2	26.1	16.2	1.6×
O4	27.8	10.0	2.8×
All ops	163.9	110.7	1.5×

Table 4: Baseline versus final configuration on the 3-worker Docker cluster, median of 3 runs; the final run includes the 18.0 s cleaned-copy write (`reports/docker-session` [2]).

in a short run. The fixed part found above also caps the speedup (Amdahl [1]). On separate nodes the contention would disappear, so the scan-dominated part should scale much closer to linearly, while the fixed part would not.

Baseline on the cluster. The baseline and the final configuration were also run on the 3-worker Docker cluster (Table 4), where shuffles cross between executor JVMs in different containers. The longest baseline stages are again the SMJ reduce stages (M2: 9 tasks, 14.6 s wall, 476 MB read; O4: 11.2 s, 222 MB read, 1.0 GB spill); after the optimisations the longest is the one-time write of the cleaned copy (16.6 s). M2's reduce stage no longer spills here, because its tasks are spread over three 1.5 GiB executors instead of sharing one 2 GiB JVM. The overall gain is smaller than locally (1.5× versus 1.8×), since the cleaned-copy write and O2's cached stages are scan- and task-bound on the oversubscribed cores, but the operations whose shuffle the broadcast removes keep the largest speedups.

6 DISCUSSION

The optional operations. Both choices exposed the bottleneck they were picked for, though not always the expected one. O2's cost was recomputation and, once that was cached, task overhead: three outputs over one aggregate made it the only operation where caching paid off, and the cache's frozen 200 partitions then became its largest cost (Sections 4.2 and 4.4). O4's cost was the join back to every trip, removed by broadcasting the bucket table; the hot-zone skew it was chosen to probe did not appear at this scale (max/median task time ≤ 1.1).

Remaining bottlenecks. After the optimisations each operation is one scan plus small stages. The scan itself is now the cost, bounded locally by CPU decode and on the raw files by their 32 row groups; the per-action scheduling cost remains as a fixed part of every run. O1 (outside the report scope) still spills 1.4 GB in a window over all trips.

Limitations. All numbers come from one laptop (`local[*]` or Docker), so shuffles never cross a network: the measured shuffle cost is a lower bound, and the cluster-size results reflect shared hardware rather than added nodes. Two series were rerun because unrelated load slowed every operation; differences below ~2 s are noise. Absolute Docker times varied by up to 1.7× between days, so every cluster number comes from one session, in which two independent sets of 3-worker runs agree within 0.5 s. Without a cluster, the 2016–2020 data was not processed, so effects that appear only at that scale (spill, skew) may be under-represented.

REFERENCES

[1] Gene M. Amdahl. 1967. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*. 483–485.

[2] Gabriel Matias Falcão. 2026. CompParLabs: Project 1 code, logs and measurement reports. https://github.com/KingPixelKP/CompParLabs/tree/main/proj1.

[3] NYC Taxi and Limousine Commission. 2026. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Yellow taxi trip records and taxi zone lookup table.

[4] The Apache Software Foundation. 2026. Apache Parquet: File Format, Row Groups and Configurations. https://parquet.apache.org/docs/.

[5] The Apache Software Foundation. 2026. Monitoring and Instrumentation (Spark 4.2.0). https://spark.apache.org/docs/latest/monitoring.html.

[6] The Apache Software Foundation. 2026. Parquet Files, Spark SQL Data Sources (Spark 4.2.0). https://spark.apache.org/docs/4.2.0/sql-data-sources-parquet.html.

[7] The Apache Software Foundation. 2026. Spark SQL Performance Tuning (Spark 4.2.0). https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html. Adaptive query execution, join strategy hints, caching.

[8] The Apache Software Foundation. 2026. Spark Standalone Mode (Spark 4.2.0). https://spark.apache.org/docs/4.2.0/spark-standalone.html.

ACKNOWLEDGEMENTS

Claude Code (Anthropic) was used as a programming and writing assistant: reviewing the implementation against the assignment, aiding in writing the measurement tooling (per-stage REST dumps, median and chart scripts), and drafting the markdown reports. All measurements were taken on the author's machine, and every number in this report traces to a log in the repository [2].

INDIVIDUAL CONTRIBUTIONS

Single-author project: Gabriel Matias Falcão (77390), 100 %.

COMMENTS AND SUGGESTIONS

Many more optimisations are possible, and exploring them is the goal of the assignment. Time and report length left several unexplored. Every optimisation is behind a feature flag, so each can be enabled on its own, and some were implemented but not reported. The DI cluster was unavailable, so all cluster results come from a local Docker standalone cluster; running the pipeline on real separate nodes and on the 2016–2020 data would be the natural next step.

ACRONYMS

AQE Adaptive Query Execution 1–3

BHJ Broadcast Hash Join 2

JVM Java Virtual Machine 3, 4

SMJ Sort-Merge Join 1–4

TLC NYC Taxi and Limousine Commission 1